CHAPTER 10

Connecting to Database

PDO • CRUD • Prepared Statements

Adding real, persistent storage to our PHP Slim REST API

SCSM2223 — Cross-Platform Application Development
Faculty of Computing • Universiti Teknologi Malaysia
Assoc. Prof. Dr Hishammuddin bin Asmuni

AT THE END OF THIS CHAPTER

Learning Objectives

C1	State what PDO is and list the benefits of a unified database abstraction layer.
C2	Differentiate between query() and prepare()/execute(), and explain why prepared statements prevent SQL injection.
C2	Describe the four CRUD operations and the corresponding SQL statements.
C3	Configure a MySQL database in Laragon, design a books table, and connect to it via PDO.
C3	Implement Create, Read, Update, Delete operations using prepared statements with positional and named placeholders.
C3	Refactor a Slim controller to delegate persistence to a Repository class.

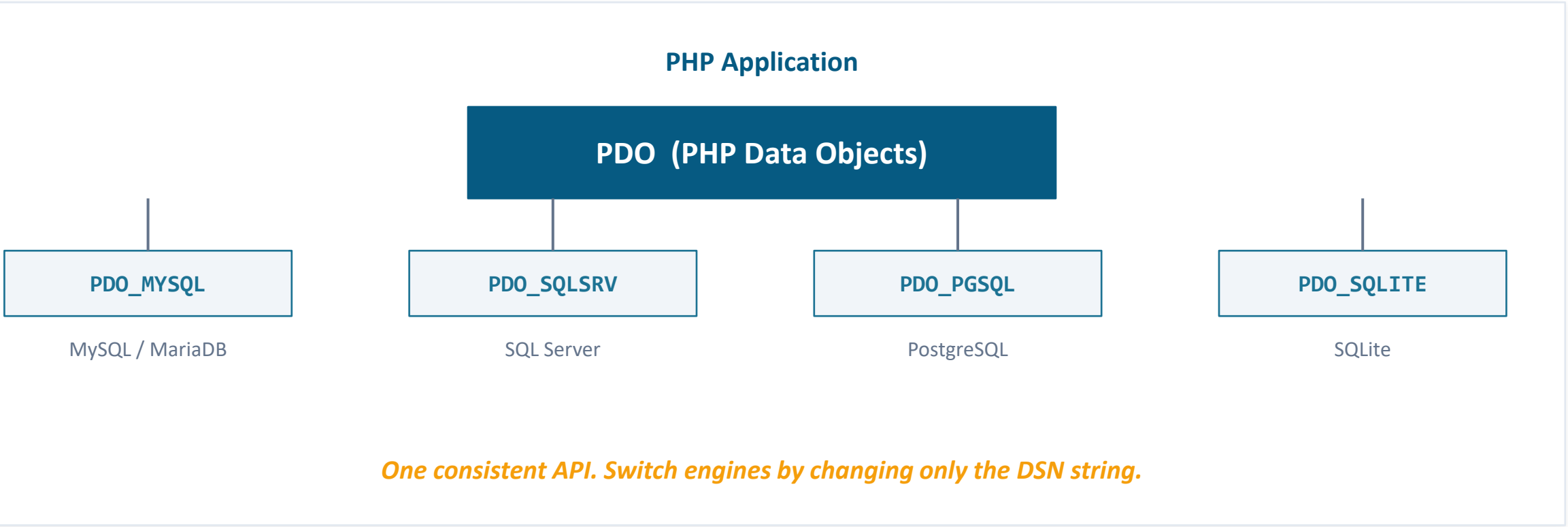
Where We Are

CHAPTER 9 — Today	CHAPTER 10 — Goal
✓ REST API in PHP Slim	✓ Persistent MySQL storage
✓ JSON requests + responses	✓ PDO with prepared statements
✓ All five CRUD verbs	✓ Repository class encapsulates SQL
✗ **Data lives in a static array**	✓ Controller stays thin
✗ **Resets on every restart**	✓ Defense against SQL injection
✗ **Useless for real users**	✓ Ready to scale beyond memory

What is PDO?

PHP Data Objects

A consistent, object-oriented API for talking to many different database engines from PHP.



One consistent API. Switch engines by changing only the DSN string.

Why Use PDO?

FEATURE	Raw mysql_*	MySQLi	PDO (this course)
Database support	MySQL only	MySQL only	12 different engines
Prepared statements	Manual escaping	Yes (mysqli_prepare)	Yes — first-class
Named placeholders	No	No (only positional)	Yes (:id , :name)
OOP API	No	Yes	Yes
Status	Removed in PHP 7	Active	Active, recommended
For our course	✗	△	✓

PDO = portable + secure + modern.

STEP 1 — CONNECT

Anatomy of a PDO Connection

DSN — Data Source Name

```
// DSN format for MySQL:
//   mysql:host=<server>;dbname=<database>;charset=utf8mb4

$dsn  = 'mysql:host=127.0.0.1;dbname=books_api;charset=utf8mb4';
$user = 'root';
$pass = '';                // Laragon's MySQL has no password by default

$pdo = new PDO($dsn, $user, $pass, [
    PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION,  // throw on errors
    PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,        // return assoc arrays
    PDO::ATTR_EMULATE_PREPARES   => false,                    // real prepared stmts
]);
```

ERRMODE_EXCEPTION

Throw a PDOException on any DB error so you can try/catch it.

FETCH_ASSOC

Make every fetch() return an associative array (no duplicate numeric keys).

EMULATE_PREPARES = false

Force MySQL to actually prepare statements on the server side.

Designing the books Table

A small, focused schema. We add timestamps for free auditing — they cost almost nothing.

```
CREATE DATABASE IF NOT EXISTS books_api
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;

USE books_api;

CREATE TABLE books (
    id          INT AUTO_INCREMENT PRIMARY KEY,
    title       VARCHAR(200) NOT NULL,
    author      VARCHAR(150) NOT NULL,
    year        SMALLINT     NOT NULL,
    genre       VARCHAR(80)  DEFAULT 'Uncategorised',
    created_at  DATETIME     DEFAULT CURRENT_TIMESTAMP,
    updated_at  DATETIME     DEFAULT CURRENT_TIMESTAMP
                              ON UPDATE CURRENT_TIMESTAMP
) ENGINE=InnoDB;

INSERT INTO books (title, author, year, genre) VALUES
  ('Clean Code',          'Robert C. Martin',  2008, 'Software Engineering'),
  ('Eloquent JavaScript', 'Marijn Haverbeke',  2018, 'Programming'),
  ('Vue.js 3 By Example', 'John Au-Yeung',     2021, 'Web Development');
```

query() vs prepare() / execute()

query() with string concatenation

```
// DO NOT do this in production!
$id = $_GET['id'];   // ← UNTRUSTED
$sql = "SELECT * FROM books WHERE id = $id";

$rows = $pdo->query($sql)->fetchAll();

// Vulnerable to SQL injection:
//   ?id=1 OR 1=1
//   ?id=1; DROP TABLE books;--
```

prepare() / execute() with placeholders

```
$id = $_GET['id'];

$stmt = $pdo->prepare(
    'SELECT * FROM books WHERE id = ?'
);
$stmt->execute([$id]);
$row = $stmt->fetch();

// Safe: $id is sent as a parameter, never
// concatenated into the SQL string.
```

Rule: any value that comes from the client must be sent as a PARAMETER, never as part of the SQL string.

WHY THIS MATTERS

SQL Injection — A Two-Minute Demo

Imagine a login form that builds the SQL by concatenation:

```
$user = $_POST['username'];
$pass = $_POST['password'];
$sql = "SELECT * FROM users WHERE username='$user' AND password='$pass'";
```

An attacker types this username:

```
username:  admin' --
password:  anything

// Resulting SQL becomes:
// SELECT * FROM users WHERE username='admin' --' AND password='anything'
//                                            ^^ everything after -- is a comment
// → logs in as 'admin' with NO password check
```

Prepared statements eliminate this entire class of attack — the data never becomes part of the query.

TWO PLACEHOLDER STYLES

Positional (?) vs Named (:name)

Positional (?)

Pass an indexed array to execute(); order matters.

```
$stmt = $pdo->prepare(
  'INSERT INTO books(title,author,year)
   VALUES(?, ?, ?)'
);
$stmt->execute([
    $title,
    $author,
    $year,
]);
```

Named (:name)

Pass an associative array; order doesn't matter — clearer for many params.

```
$stmt = $pdo->prepare(
  'INSERT INTO books(title,author,year)
   VALUES(:t, :a, :y)'
);
$stmt->execute([
  ':t' => $title,
  ':a' => $author,
  ':y' => $year,
]);
```

Fetch Modes — How to Get the Data Out

fetch()

Returns the next row. Returns false when there are no more rows.

```
$row = $stmt->fetch();
```

fetchAll()

Returns ALL rows as an array. Use only when the result set is bounded.

```
$all = $stmt->fetchAll();
```

FETCH_ASSOC

Each row is an associative array keyed by column name.

```
// set as default in PDO ATTR_DEFAULT_FETCH_MODE
```

FETCH_OBJ

Each row is a stdClass instance ($row->title , $row->id).

```
$obj = $stmt->fetch(PDO::FETCH_OBJ);
```

FETCH_CLASS

Hydrate each row directly into instances of YOUR class.

```
$stmt->fetchAll(PDO::FETCH_CLASS, Book::class);
```

fetchColumn()

Return a single column value — perfect for COUNT(*) queries.

```
$count = $stmt->fetchColumn();
```

C — CREATE

INSERT — Adding a Row

```
function createBook(PDO $pdo, array $body): array {
    $sql = 'INSERT INTO books (title, author, year, genre)
            VALUES (:title, :author, :year, :genre)';

    $stmt = $pdo->prepare($sql);
    $stmt->execute([
        ':title'  => $body['title'],
        ':author' => $body['author'],
        ':year'   => $body['year'],
        ':genre'  => $body['genre'] ?? 'Uncategorised',
    ]);

    // Get the auto-generated primary key
    $newId = (int)$pdo->lastInsertId();

    return ['id' => $newId] + $body;
}
```

UPDATE — Modifying a Row

```
function updateBook(PDO $pdo, int $id, array $body): bool {
    $sql = 'UPDATE books
               SET title  = :title,
                   author = :author,
                   year   = :year,
                   genre  = :genre
             WHERE id     = :id';

    $stmt = $pdo->prepare($sql);
    $stmt->execute([
        ':id'     => $id,
        ':title'  => $body['title'],
        ':author' => $body['author'],
        ':year'   => $body['year'],
        ':genre'  => $body['genre'],
    ]);

    return $stmt->rowCount() > 0;   // false → id did not exist
}
```

DELETE — Removing a Row

```
function deleteBook(PDO $pdo, int $id): bool {
    $stmt = $pdo->prepare('DELETE FROM books WHERE id = :id');
    $stmt->execute([':id' => $id]);
    return $stmt->rowCount() === 1;
}

// rowCount() tells you how many rows were affected.
// Use it to detect 'not found' and respond with HTTP 404.

// Defensive tip: always include a WHERE clause.
// 'DELETE FROM books'  with no WHERE = bye-bye every row.
```

MULTI-STATEMENT SAFETY

Transactions — All or Nothing

When two or more statements must succeed together, wrap them in a transaction.

```
try {
    $pdo->beginTransaction();

    $pdo->prepare('UPDATE accounts SET balance = balance - :a WHERE id = :id')
        ->execute([':a' => 100, ':id' => 1]);

    $pdo->prepare('UPDATE accounts SET balance = balance + :a WHERE id = :id')
        ->execute([':a' => 100, ':id' => 2]);

    $pdo->commit();          // both succeeded → make permanent
} catch (PDOException $e) {
    $pdo->rollBack();        // any failure → undo everything
    throw $e;
}
```

ACID guarantees: Atomicity, Consistency, Isolation, Durability — given to you for free with InnoDB.

Catching PDOException

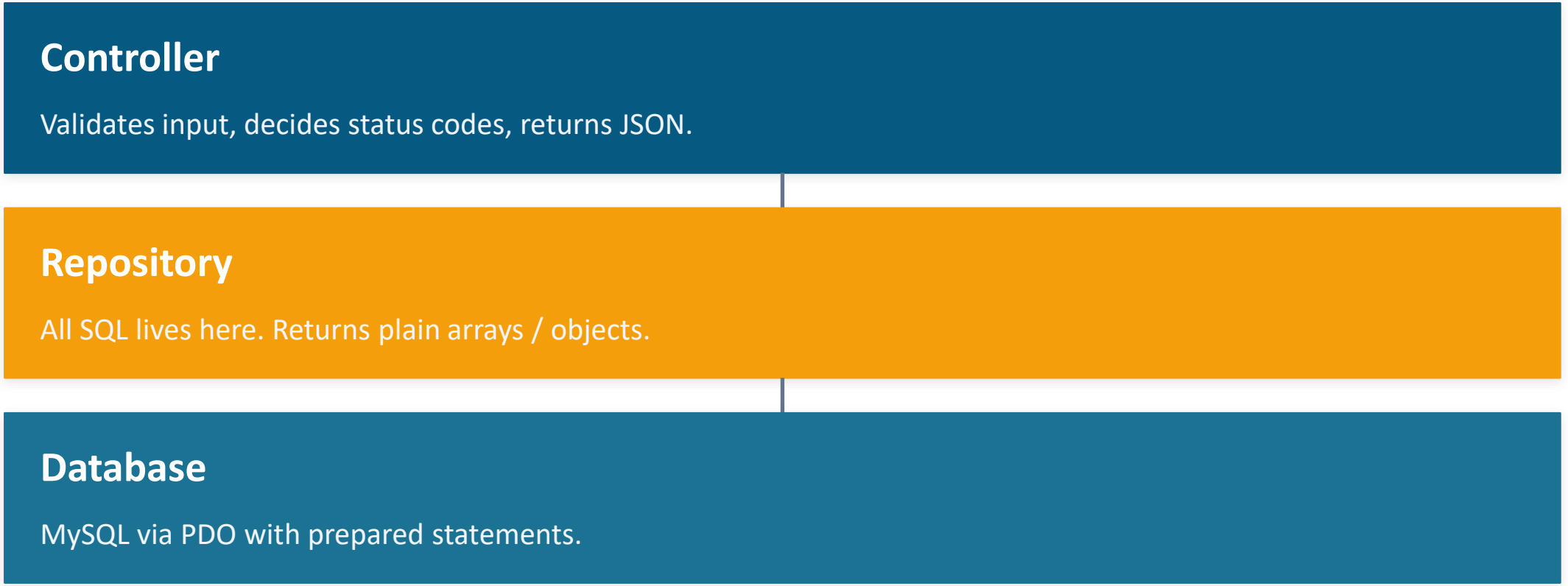
```
try {
    $pdo  = new PDO($dsn, $user, $pass, [
        PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
    ]);
    $stmt = $pdo->prepare('SELECT * FROM books WHERE id = :id');
    $stmt->execute([':id' => 99]);
    $row = $stmt->fetch();
} catch (PDOException $e) {
    // Log the FULL message internally...
    error_log('DB error: ' . $e->getMessage());
    // ...but never echo it to the client (info leak).
    http_response_code(500);
    echo json_encode(['error' => 'Database unavailable']);
}
```

Reveal nothing about your schema in error responses — log the detail server-side.

ARCHITECTURE

The Repository Pattern

Keep SQL out of your controllers. A Repository is one class that owns every query for a single resource.



WORKED EXAMPLE — PART 1

BookRepository — Read Operations

```php
<?php  // src/Repositories/BookRepository.php
namespace App\Repositories;
use PDO;

final class BookRepository {
    public function __construct(private PDO $pdo) {}

    /** @return array<int, array<string,mixed>> */
    public function all(string $q = '', int $limit = 0): array {
        $sql = 'SELECT * FROM books';
        $args = [];
        if ($q !== '') {
            $sql .= ' WHERE title LIKE :q OR author LIKE :q';
            $args[':q'] = '%' . $q . '%';
        }
        $sql .= ' ORDER BY id ASC';
        if ($limit > 0) $sql .= ' LIMIT ' . $limit;   // int-cast already

        $stmt = $this->pdo->prepare($sql);
        $stmt->execute($args);
        return $stmt->fetchAll();
    }

    public function find(int $id): ?array {
        $stmt = $this->pdo->prepare('SELECT * FROM books WHERE id = :id');
        $stmt->execute([':id' => $id]);
        return $stmt->fetch() ?: null;
    }
}
```

BookRepository — Write Operations

```
public function create(array $b): int {
    $sql = 'INSERT INTO books (title, author, year, genre)
            VALUES (:title, :author, :year, :genre)';
    $this->pdo->prepare($sql)->execute([
        ':title' => $b['title'],   ':author' => $b['author'],
        ':year'  => (int)$b['year'],':genre' => $b['genre'] ?? 'Uncategorised',
    ]);
    return (int)$this->pdo->lastInsertId();
}
public function update(int $id, array $b): bool {
    $sql = 'UPDATE books
               SET title=:title, author=:author, year=:year, genre=:genre
             WHERE id=:id';
    $stmt = $this->pdo->prepare($sql);
    $stmt->execute([
        ':id'=>$id, ':title'=>$b['title'], ':author'=>$b['author'],
        ':year'=>(int)$b['year'], ':genre'=>$b['genre'],
    ]);
    return $stmt->rowCount() > 0;
}
public function delete(int $id): bool {
    $stmt = $this->pdo->prepare('DELETE FROM books WHERE id = :id');
    $stmt->execute([':id' => $id]);
    return $stmt->rowCount() === 1;
}
```

Thin Controller, Fat Repository

```
// The controller now receives the repository via the constructor
// instead of touching SQL directly.
final class BookController {
    public function __construct(private BookRepository $books) {}

    public function index(Request $r, Response $s): Response {
        $p   = $r->getQueryParams();
        $rows = $this->books->all($p['q'] ?? '', (int)($p['limit'] ?? 0));
        return $this->json($s, ['count' => count($rows), 'data' => $rows]);
    }

    public function show(Request $r, Response $s, array $a): Response {
        $book = $this->books->find((int)$a['id']);
        return $book ? $this->json($s, $book)
                     : $this->json($s, ['error'=>'not found'], 404);
    }

    public function create(Request $r, Response $s): Response {
        $body = (array)$r->getParsedBody();
        // …validation…
        $id = $this->books->create($body);
        return $this->json($s, ['id'=>$id] + $body, 201);
    }
}
```

DO THESE EVERY TIME

PDO Best Practices

✓ **Always use prepared statements**

Even for queries you 'know' are safe.

✓ **Set ERRMODE_EXCEPTION**

Fail loudly during development; catch in production.

✓ **Use utf8mb4 charset**

Real Unicode (emoji, Chinese, BM…). utf8 in MySQL is broken.

✓ **Specify FETCH_ASSOC**

Predictable, JSON-friendly arrays out of the box.

✗ **Don't string-concat user input**

Even with addslashes(). Always parameterise.

✗ **Don't expose PDO errors**

Log them server-side; return a generic 500 to clients.

WHAT YOU NOW KNOW

Summary — Chapter 10

PDO	One unified, modern API for many databases. Use it for everything in this course.
Connection	DSN + user + pass + options. Always set ERRMODE_EXCEPTION and FETCH_ASSOC.
Prepared	prepare() + execute() = automatic protection from SQL injection.
CRUD	INSERT, SELECT, UPDATE, DELETE — each with placeholders for client data.
Repository	Keep SQL out of controllers; one class owns one resource's queries.

Next Up

Lab Hands-On — MySQL + PDO Books API

Refactor Chapter 9's API. Replace the static array with a real database, accessed safely through PDO prepared statements.

- Set up MySQL & books_api database in Laragon
- Run schema.sql to create the books table
- Build a Database connection class
- Build a BookRepository (all CRUD)
- Refactor BookController to use the repository
- Verify SQL-injection attempts are blocked